

# RELATÓRIO EXPERIMENTO 6

**Nome:** Renato Gabriel Lima da Silva

**Matrícula:** 251027871

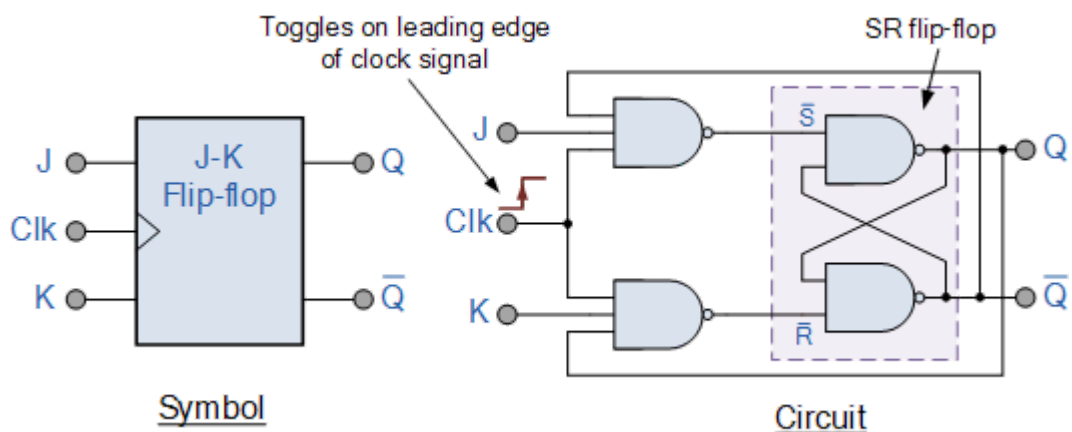
## INTRODUÇÃO

Neste experimento, iremos abordar um flip-flop do tipo JK e um circuito para realizar shift de bits (registrador de deslocamento bidirecional de 4 bits) em VHDL e simular no software ModelSim, usando a estrutura “process” da linguagem VHDL. Vamos, portanto, abordar seus conceitos e suas funcionalidades.

## TEORIA

### 1 - Flip-Flop

O modelo do flip flop JK a ser construído possui cinco entradas e uma saída. Isto é: duas entradas de controle, uma entrada de clock, duas entradas de dados, e a saída armazenando o dado. As entradas de controle forçam um estado na saída. Quando PR (preset) for 1, a saída Q deve também ser 1, independente de qualquer outro estado. Já a saída CLR (clear), ela esvazia a saída quando ativada, ou seja, quando CLR e 1, a saída e 0. A entrada de clock é usada apenas para transformar o circuito numa máquina síncrona, a qual faz com que apenas haja alterações de estado na subida de sinal (quando o clock sobe de 0 para 1). As entradas J e K servem como as entradas padrões de um flip flop desse modelo.



A tabela verdade resultante está representada na Tabela 1:

| Entradas |     |     |   |   | Saída       |
|----------|-----|-----|---|---|-------------|
| PR       | CLR | CLK | J | K | $Q_{i+1}$   |
| 1        | x   | x   | x | x | 1           |
| 0        | 1   | x   | x | x | 0           |
| 0        | 0   | ↑   | 0 | 0 | $Q_i$       |
| 0        | 0   | ↑   | 0 | 1 | 0           |
| 0        | 0   | ↑   | 1 | 0 | 1           |
| 0        | 0   | ↑   | 1 | 1 | $\bar{Q}_i$ |
| Outros   |     |     |   |   | $Q_i$       |

**Tabela 1:** Tabela verdade do JK

## 2 - Shifter

O shifter é um circuito capaz de adicionar 0 ou 1, na esquerda ou direita, de um vetor de bits pré-carregado na memória. Este circuito será projetado para um vetor de 4 bits. De acordo com as especificações requisitadas, serão necessárias três entradas de controle, três entradas de comandos, uma entrada de clock e uma saída. Essa máquina será completamente síncrona, ou seja, todas suas operações serão realizadas apenas na subida do sinal do clock (CLK).

As entradas de controle servem dois diferentes propósitos. Quando RST (reset) for 1, remove-se toda a informação guardada, ou seja, a saída Q é 0000. Já as entradas LOAD e D (data) realizam o carregamento de um vetor no circuito: quando LOAD é 1, a saída Q é D. 3 Finalmente, o funcionamento do shifter é dirigido por três variáveis. Quando DIR (direction) é 0, usa-se o valor de L (left) para realizar um shift para a esquerda. Já quando DIR é 1, usa-se o valor de R (right) para realizar um shift para a direita. A tabela verdade resultante do sistema está representada na Tabela 2:

**Tabela 2:** Tabela verdade do shifter

| Entradas |     |      |                |     |   |   | Saída  |       |       |       |
|----------|-----|------|----------------|-----|---|---|--------|-------|-------|-------|
| CLK      | RST | LOAD | D              | DIR | L | R | Q      |       |       |       |
| ↑        | 1   | x    | x              | x   | x | x | 0      | 0     | 0     | 0     |
| ↑        | 0   | 1    | $D_3D_2D_1D_0$ | x   | x | x | $D_3$  | $D_2$ | $D_1$ | $D_0$ |
| ↑        | 0   | 0    | x              | 0   | 0 | x | $Q_2$  | $Q_1$ | $Q_0$ | 0     |
| ↑        | 0   | 0    | x              | 0   | 1 | x | $Q_2$  | $Q_1$ | $Q_0$ | 1     |
| ↑        | 0   | 0    | x              | 1   | x | 0 | 0      | $Q_3$ | $Q_2$ | $Q_1$ |
| ↑        | 0   | 0    | x              | 1   | x | 1 | 1      | $Q_3$ | $Q_2$ | $Q_1$ |
| Outros   |     |      |                |     |   |   | Mantém |       |       |       |

## CÓDIGOS

### 1 - Flip-Flop

Neste experimento utilizamos a linguagem de VHDL por meio do software Modelsim para desenvolver o flip-flop, portanto segue o mesmo:

```
1  library IEEE;
2  use IEEE.std_logic_1164.all;
3
4  entity JK is
5      port (
6          J, K, clk, CLR, PR: in std_logic;
7          Q: out std_logic
8      );
9  end JK;
10
11 architecture JK_ARCH of JK is
12     signal JK : std_logic_vector(1 downto 0);
13 begin
14     JK <= J & K;
15     process(clk, CLR, PR)
16         variable Qbuf : std_logic;
17     begin
18         if PR = '1' then
19             Qbuf := '1';
20         elsif CLR = '1' then
21             Qbuf := '0';
22         elsif rising_edge(clk) then
23             case JK is
24                 when "00" => Qbuf := Qbuf;
25                 when "01" => Qbuf := '0';
26                 when "10" => Qbuf := '1';
27                 when "11" => Qbuf := not(Qbuf);
28                 when others => Qbuf := Qbuf;
29             end case;
30         end if;
31         Q <= Qbuf;
32     end process;
33 end JK_ARCH;
```

- Listagem 1: Código do flip-flop JK

### 2 - Shifter

Agora vamos analisar o código do Shifter:

```

1  |library IEEE;
2  |use IEEE.std_logic_1164.all;
3
4  entity SHIFTER is
5      port (
6          clk, rst, load, DIR, L, R: in std_logic;
7          D: in std_logic_vector(3 downto 0);
8          Q: out std_logic_vector(3 downto 0)
9      );
10 end SHIFTER;
11
12 architecture SHIFTER_ARCH of SHIFTER is
13 begin
14     process(clk)
15         variable Qbuf : std_logic_vector(3 downto 0);
16     begin
17         if rising_edge(clk) then
18             if rst = '1' then
19                 Qbuf := "0000";
20             elsif load = '1' then
21                 Qbuf := D;
22             elsif DIR = '0' then
23                 Qbuf := Qbuf(2 downto 0) & L;
24             else
25                 Qbuf := R & Qbuf(3 downto 1);
26             end if;
27         end if;
28         Q <= Qbuf;
29     end process;
30 end SHIFTER_ARCH;

```

- Listagem 2: Código do Shifter.

## SIMULAÇÃO E ANÁLISE

### 1 - Flip-Flop

Para a simulação, usa-se de um testbench e um top module para controlar os testes. O código do testbench utilizado com o flip flop JK segue na Listagem 3, enquanto o top module está descrito na Listagem 4. O circuito é colocado em uma série de testes sequenciais. Inicialmente, no teste 1, verifica-se se a entrada PR funciona, e no teste 2 se a entrada CLR funciona. Para o teste 3 checa-se se a combinação 00 para JK é válida, no teste 4 a combinação 01, e assim por diante, até o teste 6. Verifica-se, pelas Figuras 1 que não ocorreu nenhum erro durante a simulação. A saída DUT\_out representa a saída Q do circuito.

```

1  library IEEE;
2  use IEEE.std_logic_1164.all;
3
4  entity TBJK is
5      port (
6          DUT_out: in std_logic;
7          J, K, clk, CLR, PR: out std_logic
8      );
9  end TBJK;
10
11 architecture TBJK_ARCH of TBJK is
12 begin
13     process
14     begin
15         report "Iniciando teste..." severity NOTE;
16
17         J <= '0';
18         K <= '0';
19         CLR <= '0';
20         PR <= '1';
21         clk <= '0';
22         wait for 250 ns;
23         clk <= '1';
24         wait for 250 ns;
25         assert (DUT_out = '1') report "Falha no Teste 1." severity ERROR;
26
27         PR <= '0';
28         CLR <= '1';
29         clk <= '0';
30         wait for 250 ns;
31         clk <= '1';
32         wait for 250 ns;
33         assert (DUT_out = '0') report "Falha no Teste 2." severity ERROR;
34
35         CLR <= '0';
36         J <= '0';
37         K <= '0';
38         clk <= '0';
39         wait for 250 ns;
40         clk <= '1';
41         wait for 250 ns;
42         assert (DUT_out = '0') report "Falha no Teste 3." severity ERROR;
43
44         J <= '1';
45         K <= '0';
46         clk <= '0';

```

```

47         wait for 250 ns;
48         clk <= '1';
49         wait for 250 ns;
50         assert (DUT_out = '1') report "Falha no Teste 4." severity ERROR;
51
52         J <= '0';
53         K <= '1';
54         clk <= '0';
55         wait for 250 ns;
56         clk <= '1';
57         wait for 250 ns;
58         assert (DUT_out = '0') report "Falha no Teste 5." severity ERROR;
59
60         J <= '1';
61         K <= '1';
62         clk <= '0';
63         wait for 250 ns;
64         clk <= '1';
65         wait for 250 ns;
66         assert (DUT_out = '1') report "Falha no Teste 6." severity ERROR;
67
68         report "Teste finalizado." severity NOTE;
69         wait;
70     end process;
71 end TBJK_ARCH;

```

- Listagem 3: Código do testbench.

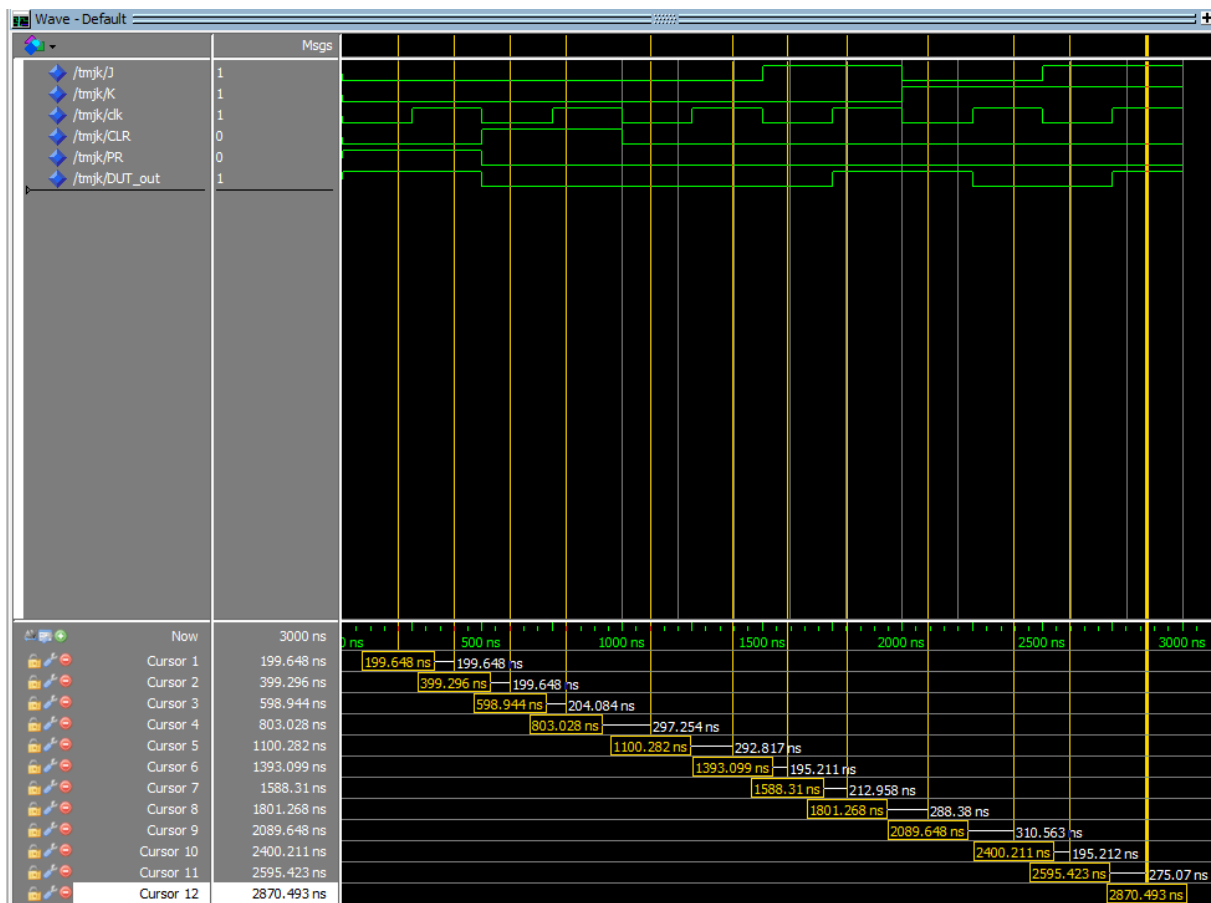
```

1  library IEEE;
2  use IEEE.std_logic_1164.all;
3
4  entity TMJK is
5  end TMJK;
6
7  architecture TMJK_ARCH of TMJK is
8      component JK is
9          port (
10              J, K, clk, CLR, PR: in std_logic;
11              Q: out std_logic
12          );
13      end component;
14
15      component TBJK is
16          port (
17              DUT_out: in std_logic;
18              J, K, clk, CLR, PR: out std_logic
19          );
20      end component;
21
22      signal J, K, clk, CLR, PR : std_logic;
23      signal DUT_out : std_logic;
24
25  begin
26      INT1 : JK port map(J, K, clk, CLR, PR, DUT_out);
27      INT2 : TBJK port map(DUT_out, J, K, clk, CLR, PR);
28  end TMJK_ARCH;

```

- Listagem 4: Código do top module

Agora vamos simular:



- Figura 1: Simulação com zoom completo do flip-flop JK

## 2 - Shifter

Para a simulação, usa-se de um testbench e um top module para controlar os testes. O código do testbench segue na Listagem 5, enquanto o top module está descrito na Listagem 6. O circuito é colocado numa série de testes sequenciais. Primeiramente, coloca-se a entrada reset como 1, e verifica-se se a saída está com o vetor 0000. Para o teste 2, introduz-se o vetor 0110 na saída por meio das entradas load e D. Segue-se, então, uma sequência de testes para verificar a funcionalidade das entradas DIR, L e R. Empurra-se um 0 e um 1 à esquerda do vetor para os testes 3 e 4. E, finalmente, um 0 e um 1 à direita do vetor para os testes 5 e 6, finalizando-se os testes.

```

1  library IEEE;
2  use IEEE.std_logic_1164.all;
3
4  entity TBSH is
5      port (
6          DUT_out: in std_logic_vector(3 downto 0);
7          clk, rst, load, DIR, L, R: out std_logic;
8          D: out std_logic_vector(3 downto 0)
9      );
10 end TBSH;
11
12 architecture TBSH_ARCH of TBSH is
13 begin
14     process
15     begin
16         report "Iniciando teste..." severity NOTE;
17
18         clk <= '0';
19         rst <= '1';
20         load <= '0';
21         D <= "0000";
22         DIR <= '0';
23         L <= '0';
24         R <= '0';
25         wait for 250 ns;
26         clk <= '1';
27         wait for 250 ns;
28         assert (DUT_out = "0000") report "Falha no Teste 1." severity ERROR;
29
30         clk <= '0';
31         rst <= '0';
32         load <= '1';
33         D <= "0110";
34         wait for 250 ns;
35         clk <= '1';
36         wait for 250 ns;
37         assert (DUT_out = "0110") report "Falha no Teste 2." severity ERROR;
38
39         clk <= '0';
40         load <= '0';
41         DIR <= '0';
42         L <= '0';
43         wait for 250 ns;
44         clk <= '1';
45         wait for 250 ns;
46         assert (DUT_out = "1100") report "Falha no Teste 3." severity ERROR;
47
48         clk <= '0';
49         DIR <= '0';

```

```

50         L <= '1';
51         wait for 250 ns;
52         clk <= '1';
53         wait for 250 ns;
54         assert (DUT_out = "1001") report "Falha no Teste 4." severity ERROR;
55
56         clk <= '0';
57         DIR <= '1';
58         R <= '0';
59         wait for 250 ns;
60         clk <= '1';
61         wait for 250 ns;
62         assert (DUT_out = "0100") report "Falha no Teste 5." severity ERROR;
63
64         clk <= '0';
65         DIR <= '1';
66         R <= '1';
67         wait for 250 ns;
68         clk <= '1';
69         wait for 250 ns;
70         assert (DUT_out = "1010") report "Falha no Teste 6." severity ERROR;
71
72         report "Teste finalizado." severity NOTE;
73         wait;
74     end process;
75 end TBSH_ARCH;

```

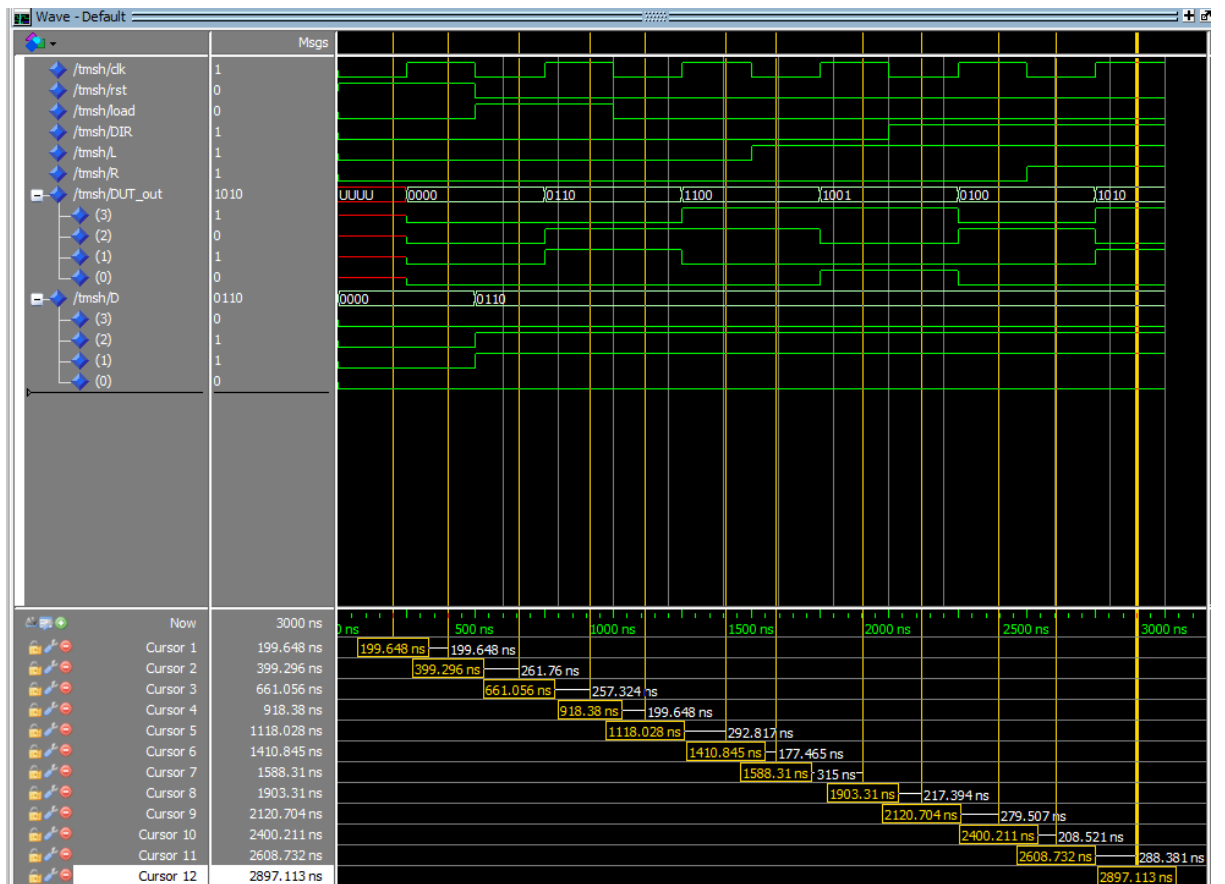
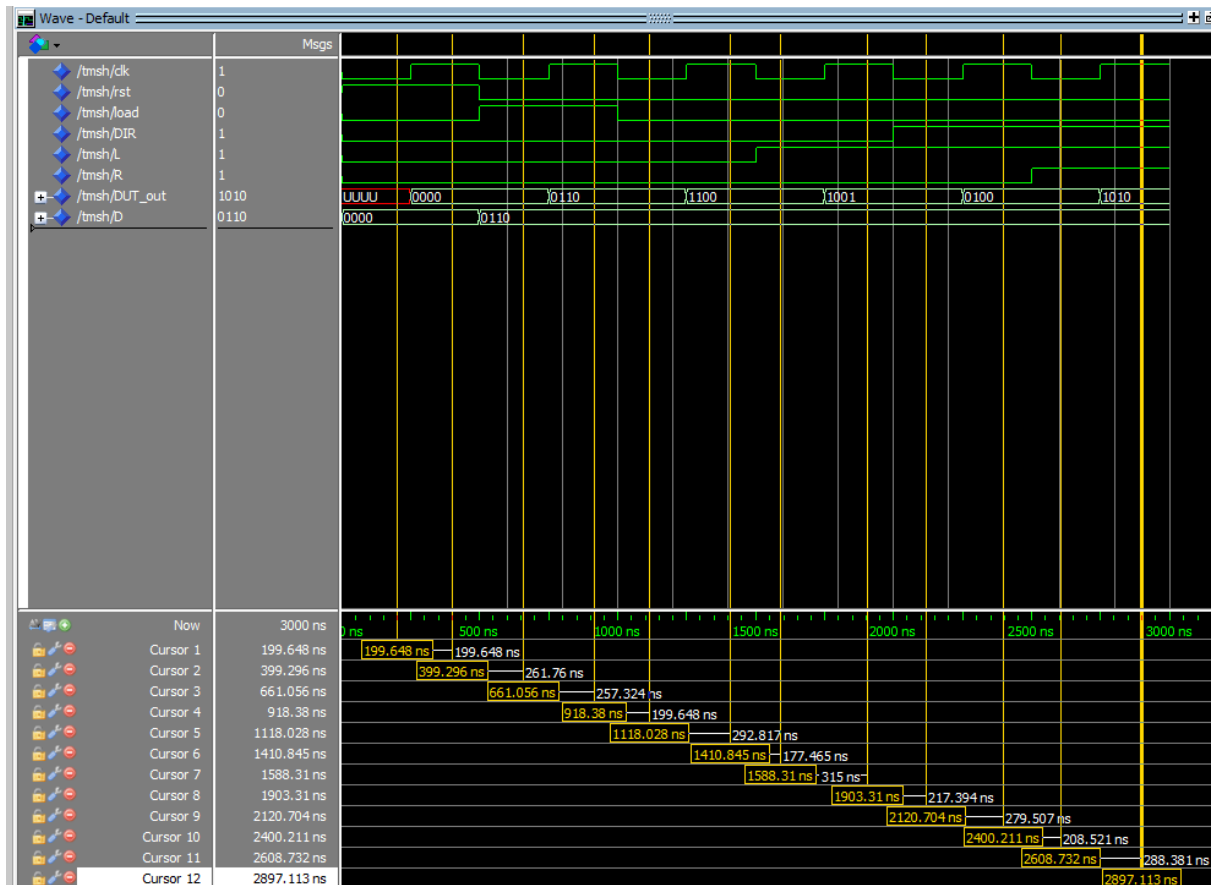
- listagem 5: código testbench do shifter

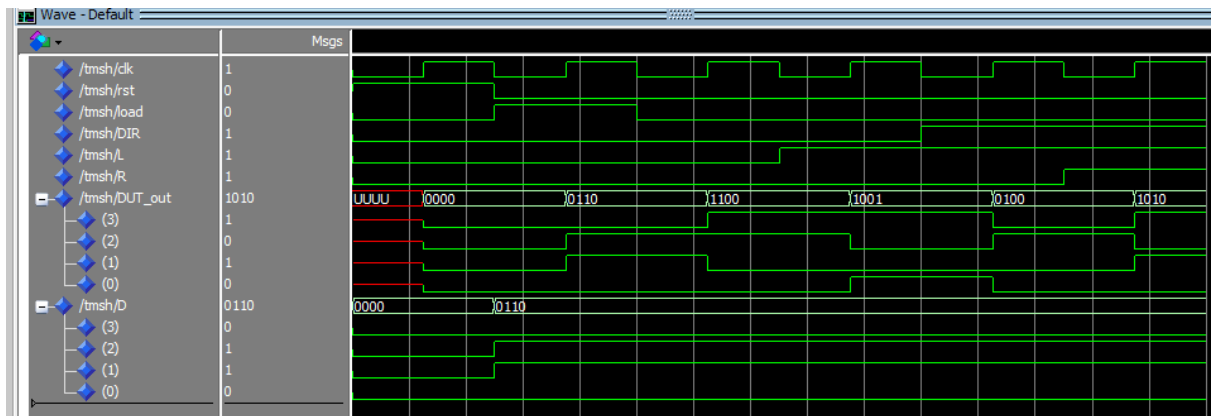
```

1  library IEEE;
2  use IEEE.std_logic_1164.all;
3
4  entity TMSH is
5  end TMSH;
6
7  architecture TMSH_ARCH of TMSH is
8      component SHIFTER is
9          port (
10              clk, rst, load, DIR, L, R: in std_logic;
11              D: in std_logic_vector(3 downto 0);
12              Q: out std_logic_vector(3 downto 0)
13          );
14      end component;
15
16      component TBSH is
17          port (
18              DUT_out: in std_logic_vector(3 downto 0);
19              clk, rst, load, DIR, L, R: out std_logic;
20              D: out std_logic_vector(3 downto 0)
21          );
22      end component;
23
24      signal clk, rst, load, DIR, L, R : std_logic;
25      signal DUT_out, D : std_logic_vector(3 downto 0);
26
27  begin
28      INT1 : SHIFTER port map(clk, rst, load, DIR, L, R, D, DUT_out);
29      INT2 : TBSH port map(DUT_out, clk, rst, load, DIR, L, R, D);
30  end TMSH_ARCH;

```

- Listagem 6: código do Top module





• Figura 2: Simulação do shifter

## COMPILAÇÃO

Os códigos gerados anteriormente foram submetidos a uma compilação com o intuito de garantir o seu funcionamento:

|           |   |      |   |                      |
|-----------|---|------|---|----------------------|
| tm_sh.vhd | ✓ | VHDL | 5 | 05/21/26 02:57:16 PM |
| e2.vhd    | ✓ | VHDL | 1 | 05/21/26 02:55:59 PM |
| tb_sh.vhd | ✓ | VHDL | 3 | 05/21/26 02:56:12 PM |
| tm_jk.vhd | ✓ | VHDL | 4 | 05/21/26 02:57:11 PM |
| tb_jk.vhd | ✓ | VHDL | 2 | 05/21/26 02:56:06 PM |
| e1.vhd    | ✓ | VHDL | 0 | 05/21/26 02:55:54 PM |

```

Library x Project x
Transcript
# Reading C:/altera/13.0spl/modelsim_ase/tcl/vsim/p
# Loading project exp6
# Compile of e1.vhd was successful.
# Compile of e2.vhd was successful.
# Compile of tb_jk.vhd was successful.
# Compile of tb_sh.vhd was successful.
# Compile of tm_jk.vhd was successful.
# Compile of tm_sh.vhd was successful.
# 6 compiles, 0 failed with no errors.

```

• Figura 2: Compilação dos códigos

Como visto na figura 2, nenhum código tem algum erro de sintaxe.

## CONCLUSÃO

Portanto, neste experimento conseguimos com êxito implementar os dois circuitos chegando a conclusão que estão funcionando como o esperado, tanto o flip flop JK quanto o shifter. Não foram encontrados erros ou divergências neste experimento.